

Software Development:

- Software Development = Process of designing, building, testing, and maintaining software applications.
- It follows a structured approach to ensure quality, cost-effectiveness, and timely delivery.
- In practice, this is managed using Software Development Life Cycle (SDLC).

Phases of the Software Development Life Cycle:

1. Requirement Analysis

- Requirement gathering techniques (Interviews, Surveys, Prototypes)
- Functional vs Non-functional requirements
- Feasibility study (technical, operational, economic)

2. System Design

- High-level design (architecture, database design, data flow diagrams)
- Low-level design (module specifications, algorithms)
- Tools (UML diagrams, ER diagrams)

3. Implementation (Coding)

- Choice of programming languages
- Coding standards & guidelines
- Code reviews and version control

4. Testing

- Types of testing: Unit, Integration, System, Acceptance
- Black-box vs White-box testing
- Automation testing tools

5. Deployment

- Deployment strategies (Big bang, Incremental, Parallel, Phased)
- Continuous Deployment & DevOps

6. Maintenance

- Corrective, Adaptive, Perfective, Preventive maintenance
- Software evolution (Lehman's laws)

Software Development Life Cycle (SDLC) Models

SDLC Models describe how the different phases of software development are carried out.

1. Waterfall Model:

- Oldest and simplest model.
- Each phase flows into the next like a waterfall.

Process:

Requirement → Design → Implementation → Testing → Deployment → Maintenance

Advantages:

- Easy to understand and manage.
- Best for small projects with well-defined requirements.
- Clear milestones and documentation.

Disadvantages:

- Inflexible: once a phase is completed, hard to go back.
- Not suitable for projects where requirements change frequently.
- Late discovery of issues (during testing).

Use Case:

- Payroll system, banking software, government projects.

2. Iterative Model

- Develop software **in small iterations (cycles)**.
- Each iteration adds more features until the full system is ready.

Advantages:

- Early delivery of partial working system.
- Easier to fix problems in earlier iterations.
- Suitable for projects where requirements are not fully clear.

Disadvantages:

- Requires good planning.
- Cost may increase due to repeated cycles.

Use Case:

- Large systems where user feedback is required at each stage.

3. Incremental Model

- Similar to iterative model, but software is built **piece by piece** (increments).
- Each increment = fully working software module.

Advantages:

- Users get useful parts of the system earlier.
- Easier testing and debugging.
- Flexible and less risky.

Disadvantages:

- Needs proper design from the beginning.
- Final integration may be complex.

Use Case:

- E-commerce websites, online booking systems.

4. V-Model (Verification and Validation Model)

- Extension of Waterfall.
- Every development phase has a **corresponding testing phase**.

Structure:

- Left side: Requirement → Design → Coding
- Right side: Testing (Unit test, Integration test, System test, Acceptance test)

Advantages:

- Strong focus on testing.
- Defects can be caught early.

Disadvantages:

- Same rigidity as Waterfall.
- Costly if requirements change.

Use Case:

- Safety-critical systems (medical devices, defense software).

5. Spiral Model

- Combines **Iterative Model + Risk Analysis**.
- Development is done in spirals (loops), each loop has 4 steps:
 1. Planning
 2. Risk Analysis
 3. Engineering (development + testing)
 4. Evaluation

Advantages:

- Good for large, complex, and high-risk projects.
- Risk is managed at every stage.
- Flexible to requirement changes.

Disadvantages:

- Expensive and complex.
- Requires skilled risk management.

Use Case:

- Aerospace software, banking systems, large defense projects.

6. Big Bang Model

- No structured process. Developers start coding immediately.

Advantages:

- Very simple, less planning required.
- Best for small projects or academic purposes.

Disadvantages:

- Very risky, no documentation.
- Not suitable for large projects.

Use Case:

- Small experimental projects, student projects.

7. Agile Model

- Modern, **iterative and incremental** model.
- Focus on **customer collaboration, adaptive planning, and continuous delivery**.
- Work is divided into short cycles called **sprints** (1–4 weeks).

Principles:

- Individuals and interactions over processes.
- Working software over documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

Advantages:

- Highly flexible, customer feedback integrated quickly.
- Early and continuous delivery of working software.
- Best for dynamic projects.

Disadvantages:

- Requires experienced team.
- Customer must be available throughout.
- Less focus on documentation.

Use Case:

- Mobile apps, web applications, startups, rapidly changing environments.

8. Prototype Model

- A **working prototype** (sample) of software is built first.
- Users give feedback, then final system is developed.

Advantages:

- Helps users understand system requirements.
- Reduces risk of wrong requirements.

Disadvantages:

- Prototypes may cause misunderstanding (users may think it's the final product).
- Time consuming.

Use Case:

- Projects where requirements are unclear (e.g., new product ideas).

9. RAD (Rapid Application Development) Model

- Based on **prototyping and iterative delivery**.
- Emphasis on quick development using tools, code reusability, and automation.

Advantages:

- Faster delivery.
- Encourages user involvement.

Disadvantages:

- Not suitable for large, complex systems.
- Requires skilled developers and tools.

Use Case:

- Small-to-medium projects with short deadlines.